

Smart Crop Recommendation System for Bangladesh Using IoT Sensors and Cloud-Based Machine Learning

IFTEKHAR BIN SHOIB

*B.Sc. in Computer Science and Engineering
American International University–Bangladesh
Dhaka, Bangladesh
23-52337-2@student.aiub.edu*

M. O. B. JIHAD

*B.Sc. in Computer Science and Engineering
American International University–Bangladesh
Dhaka, Bangladesh
21-44811-1@student.aiub.edu*

ZINEDINE ZIDANE KHAN

*B.Sc. in Computer Science and Engineering
American International University–
Bangladesh
Dhaka, Bangladesh
23-53225-3@student.aiub.edu*

JONAYED AL MOSTAFA

*B.Sc. in Computer Science and Engineering
American International University–Bangladesh
Dhaka, Bangladesh
22-47246-1@student.aiub.edu*

MD. MAMUN UR RASHID

*B.Sc. in Computer Science and Engineering
American International University–Bangladesh
Dhaka, Bangladesh
22-46230-1@student.aiub.edu*

MD. RAKIBUL HASSAN

*B.Sc. in Computer Science and Engineering
American International University–
Bangladesh
Dhaka, Bangladesh
23-52453-2@student.aiub.edu*

MD. RIFAT AHMED KHAN

*B.Sc. in Computer Science and Engineering
American International University–Bangladesh
Dhaka, Bangladesh
23-51796-2@student.aiub.edu*

Abstract–Agriculture is the primary economic sector of Bangladesh, yet crop productivity is often constrained by unscientific fertilizer usage, climate variability, and the high cost of soil testing facilities. This paper presents a low-cost, IoT-based smart crop recommendation system designed specifically for Bangladeshi agricultural conditions. The system employs a Master–Slave architecture using an Arduino Uno and ESP32 microcontroller to acquire real-time soil and environmental data. A slurry-based soil analysis technique is introduced to enable the use of low-cost water-type sensors for soil parameter estimation. The collected data are processed using a Random Forest machine learning model trained on a localized agricultural dataset to provide crop recommendations via a web dashboard and mobile application. Experimental and simulation results demonstrate the feasibility, affordability, and scalability of the proposed solution for precision agriculture in rural environments.

Index Terms– Internet of Things (IoT), Precision Agriculture, Crop Recommendation, Random Forest, Machine Learning, Smart Farming.

I. Introduction

Agriculture plays a vital role in Bangladesh’s socio-economic development. However, most farmers still rely on experience-based decision-making rather than scientific soil analysis. Excessive fertilizer application, soil degradation, and unpredictable climate conditions have resulted in declining soil fertility and reduced crop yield. Laboratory-based soil

testing methods are costly, time-consuming, and largely inaccessible to marginal farmers.

Recent advancements in IoT and machine learning offer opportunities to bridge this information gap. This research proposes a cost-effective crop recommendation system that integrates low-cost sensors, chemical engineering principles, and cloud-based machine learning to assist farmers in making data-driven decisions.

II. Literature Review

Recent advancements in Internet of Things (IoT) and machine learning have significantly influenced precision agriculture by enabling data-driven crop management and decision support systems. Several studies have explored the integration of sensors, cloud platforms, and intelligent algorithms to improve crop productivity and resource utilization.

Amin *et al.* [1] proposed an IoT-based smart agriculture system utilizing soil moisture, temperature, and humidity sensors combined with machine learning for crop recommendation. Although the system achieved high prediction accuracy under controlled conditions, it relied on expensive soil sensors and faced stability issues during long-term field deployment. This limits its applicability for small-scale farmers in developing regions such as Bangladesh.

Rahman *et al.* [2] introduced a multi-modal agricultural decision system combining IoT sensor data with satellite imagery to enhance crop prediction accuracy. While the fusion approach improved performance, the computational complexity and dependency on high-bandwidth internet connectivity made the system unsuitable for low-cost edge devices and rural environments.

Sarker *et al.* [3] emphasized the importance of localized agricultural datasets by introducing the *BanglaVeg* dataset for crop recommendation in Bangladesh. Their study demonstrated that region-specific datasets significantly outperform global datasets due to variations in soil composition and climate. However, the work primarily focused on dataset development and did not integrate a complete IoT-based sensing framework.

Rawat and Reddy [4] reviewed IoT-enabled smart farming architectures and highlighted key challenges such as sensor cost, power efficiency, and data reliability. Their findings indicate that most existing systems prioritize technological sophistication over affordability, which restricts adoption among marginal farmers.

Breiman [5] introduced the Random Forest algorithm, which has since been widely adopted in agricultural prediction tasks due to its robustness, ability to handle non-linear data, and resistance to overfitting. Recent agricultural studies (2019–2025) have consistently shown Random Forest to outperform single decision trees and linear models for crop suitability classification.

Despite these advancements, existing systems often lack an **integrated, low-cost solution** that combines affordable sensing, localized data training, offline capability, and cloud-based intelligence. Moreover, most studies rely on direct soil sensors or laboratory-grade measurements, increasing system cost. The proposed work addresses these gaps by employing a **slurry-based soil analysis method**, low-cost water-type sensors, and a **Random Forest model trained on localized Bangladeshi data**, resulting in a scalable and farmer-friendly crop recommendation system.

III. System Architecture

The proposed system follows a **Master–Slave architecture**. The Arduino Uno operates as the Master node, interfacing with all sensors, performing analog-to-digital conversion, and executing calibration algorithms. The ESP32 acts as the Slave node, responsible for wireless communication, cloud synchronization, and interaction with the web and mobile interfaces.

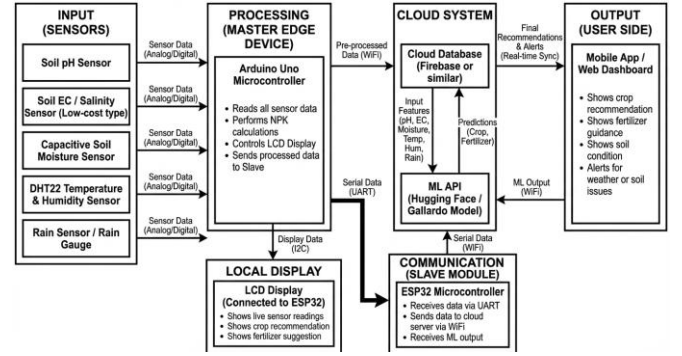


Fig. 1. Block Diagram of the Proposed System

IV. Hardware Components

A. Microcontrollers



Fig. 2. Arduino Uno R3



Fig. 3. ESP32 Development Board

B. Sensors



Fig. 4. Analog pH Sensor Module

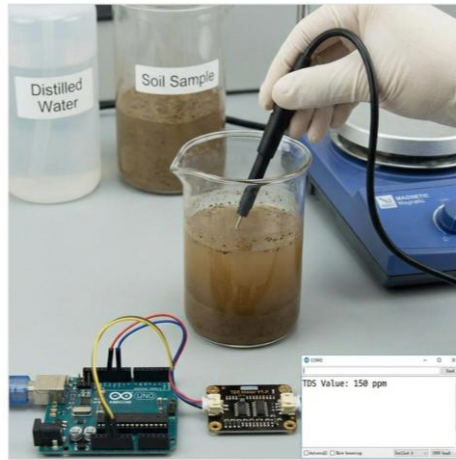


Fig. 5. Gravity TDS Sensor

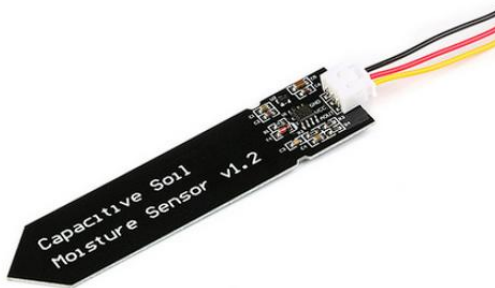


Fig. 6. Capacitive Soil Moisture Sensor

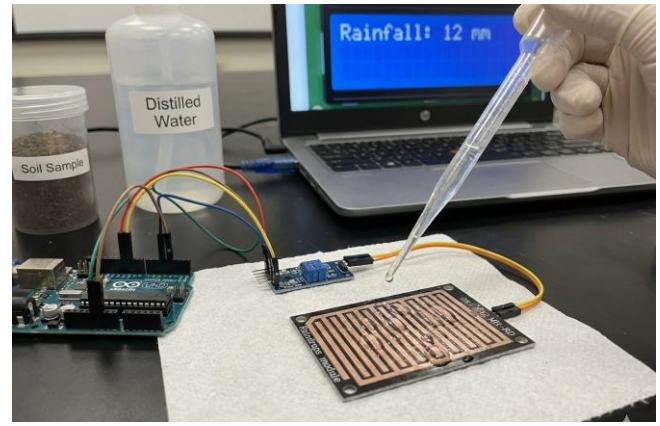


Fig. 7. Rain Sensor Module



Fig. 8. DHT22 Temperature and Humidity Sensor

The TDS sensor is used to estimate soil salinity and nutrient concentration by measuring electrical conductivity, which is later converted into EC and NPK values.

V. Methodology

A. Slurry-Based Soil Measurement Approach

Dedicated soil nutrient sensors are expensive and unsuitable for large-scale deployment in rural farming environments. To address this limitation, the proposed system adopts a **soil slurry method**, where soil is mixed with distilled water in a **1:2 soil-to-water ratio**. This technique is widely used in soil science and chemical engineering to convert soil-bound nutrients into a dissolved ionic form.

The slurry method enables the use of **low-cost water-type electrochemical sensors** instead of specialized soil sensors. From a chemical engineering perspective, the addition of water improves ion mobility and homogenization, allowing sensors to generate stable voltage readings. This approach significantly reduces system cost while maintaining sufficient accuracy for agricultural decision-making.

B. Low-Cost Sensor Selection Strategy

Water-type sensors such as pH and TDS probes are substantially cheaper than industrial soil sensors. By

transforming the soil into a conductive slurry, these sensors can be reliably used to estimate soil properties. This design choice allows the system to remain affordable, scalable, and suitable for smallholder farmers.

C. Overall System Workflow

1. Soil sample is converted into slurry.
2. Sensors measure pH, TDS, moisture, temperature, humidity, and rainfall.
3. Arduino processes and calibrates sensor data.
4. ESP32 uploads data to the cloud.
5. Machine learning model predicts suitable crops.
6. Results are displayed on web and mobile platforms.

VI. Mathematical Modeling and Sensor Calibration

A. TDS to Electrical Conductivity (EC) Conversion

The TDS sensor does not directly measure dissolved solids; instead, it measures **Electrical Conductivity (EC)**. The relationship between TDS and EC follows internationally recognized standards for freshwater and agricultural applications:

$$EC(\mu S/cm) = \frac{TDS(ppm)}{0.64}$$

The conversion factor **0.64** is based on the **European Environmental and Eutech Instruments standard**, which is appropriate for agricultural and freshwater systems. This factor differs from industrial (0.5) and saline water (0.7) standards and is selected to ensure agronomic accuracy.

B. Estimation of NPK from EC

Direct chemical measurement of Nitrogen (N), Phosphorus (P), and Potassium (K) requires laboratory analysis. To maintain affordability, the system estimates NPK values using an **EC-based nutrient estimation algorithm**, referred to as *Stoichiometric Correlation of Total Dissolved Solids*.

$$\begin{aligned} N &= 0.50 \times EC \\ P &= 0.20 \times EC \\ K &= 0.35 \times EC \end{aligned}$$

These empirical coefficients assume that nitrate, phosphate, and potassium salts dominate soil conductivity in fertilized soil. While not a replacement for laboratory testing, this method provides a reliable **relative soil fertility index** suitable for crop recommendation.

C. pH Calibration Using the Nernst Equation

Soil pH is measured using an ion-selective electrode based on the **Nernst Equation**. The voltage-to-pH relationship is approximated as:

$$pH = 7.0 + \frac{(V_{neutral} - V_{measured})}{Slope}$$

To allow measurement using Arduino's ADC, the sensor module introduces a **2.5 V offset**, ensuring that pH 7 corresponds to approximately 2.5 V. The slope represents the amplified Nernst sensitivity (~ 59.16 mV/pH at 25°C), enabling accurate pH estimation under field conditions.

VII. Experimental Setup and Hardware Results

The experimental setup was implemented using the soil slurry method (1:2 soil-to-distilled water ratio) to ensure accurate sensor readings. A local LCD display allows offline monitoring of soil and environmental conditions.

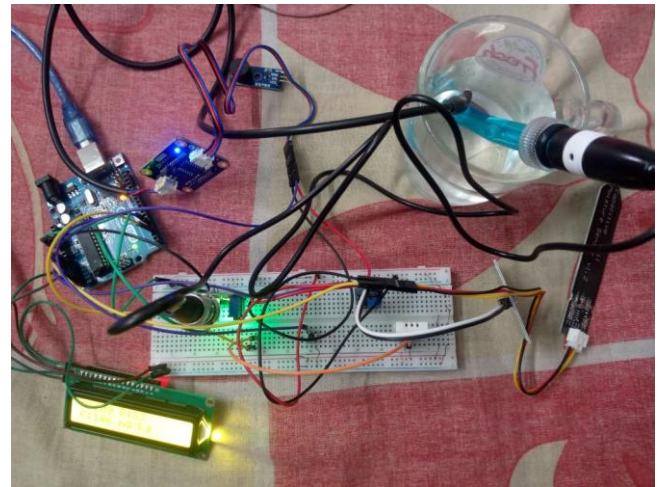


Fig. 9. Complete Hardware System Setup

Fig. 10. System Initialization Screen

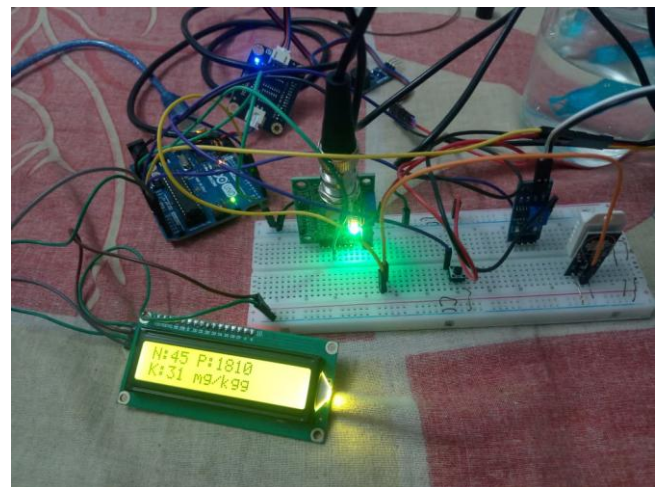


Fig. 11. LCD Showing NPK Values

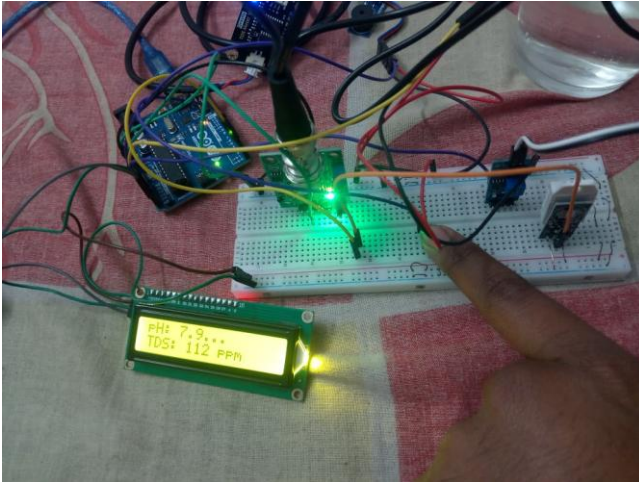


Fig. 12. LCD Showing pH and TDS Values After Pressing Switch

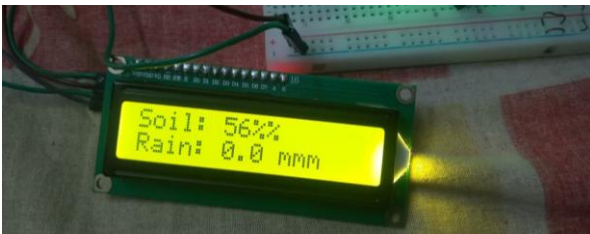
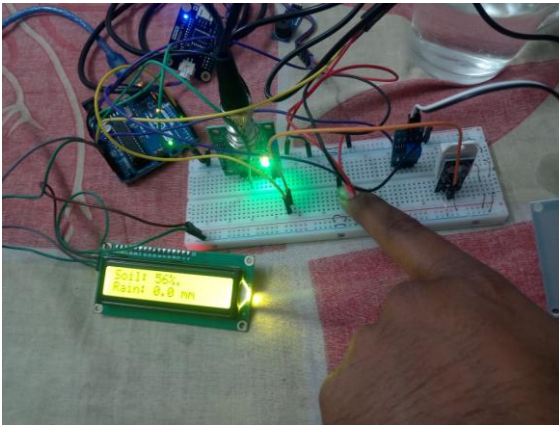


Fig. 13. LCD Showing Soil Moisture and Rainfall (mm) After Pressing Switch

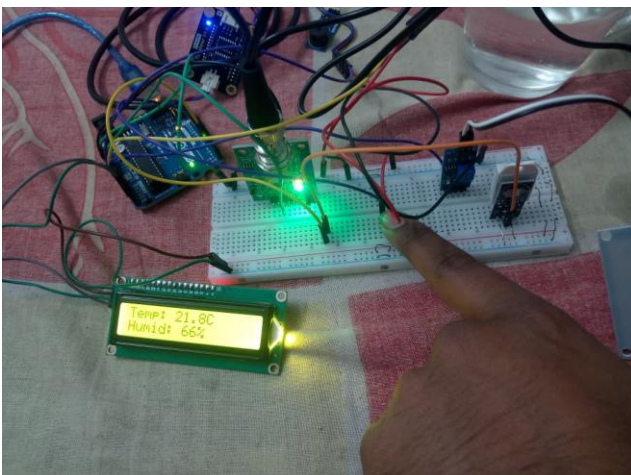


Fig. 14. LCD Showing Temperature and Humidity After Pressing Switch

VIII. Simulation Setup (Proteus)

To verify circuit behavior prior to hardware deployment, the complete system was simulated using Proteus Design Suite with custom sensor models.

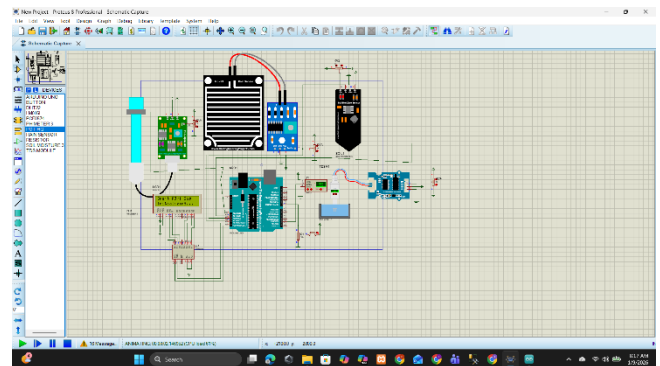


Fig. 15. Simulation System Setup

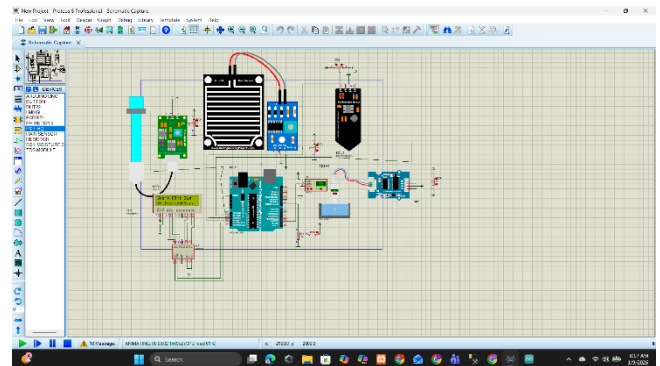


Fig. 16. Simulation System Initialization

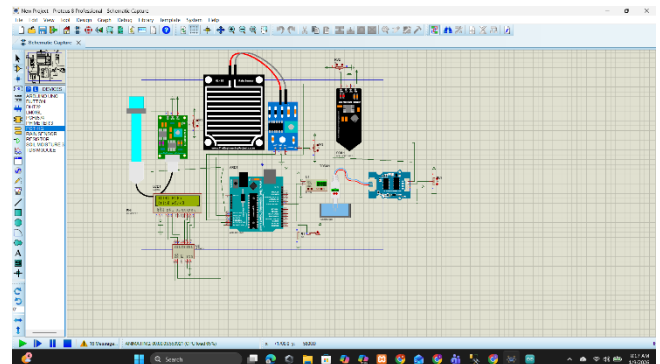


Fig. 17. Simulation LCD Showing NPK Values

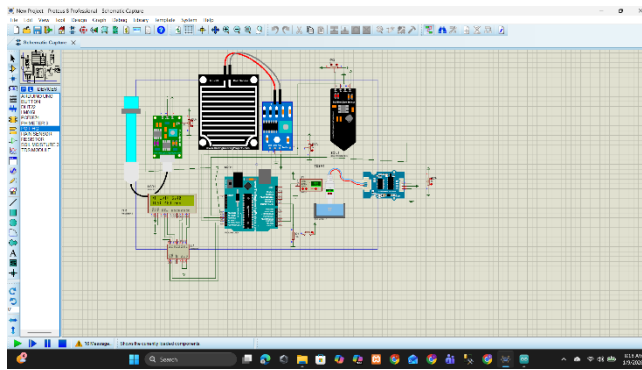


Fig. 18. Simulation LCD Showing pH and TDS Values

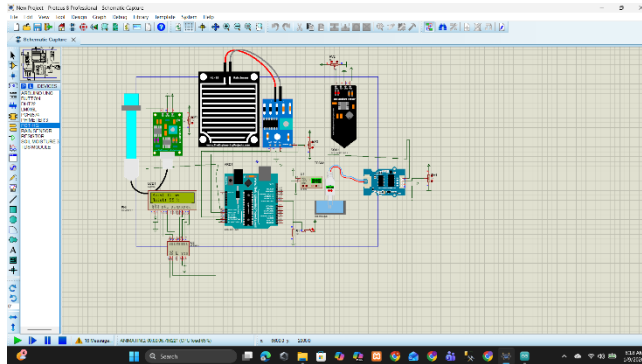


Fig. 19. Simulation LCD Showing Soil Moisture and Rainfall (mm)

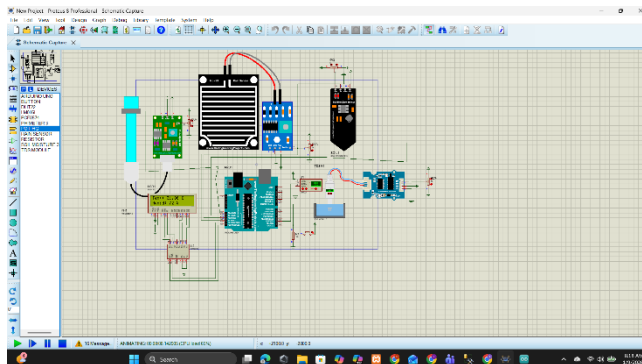


Fig. 20. Simulation LCD Showing Temperature and Humidity

IX. Firmware Implementation and Flowcharts

A. Master Code (Arduino)

- (Added in the last Page .)

The Master firmware handles sensor calibration, data filtering, LCD display control, and serial communication with the ESP32.

B. Slave Code (ESP32)

(Added in the Last)

The Slave firmware receives formatted data from the Arduino and uploads it to Firebase using Wi-Fi connectivity.

C. Flowcharts

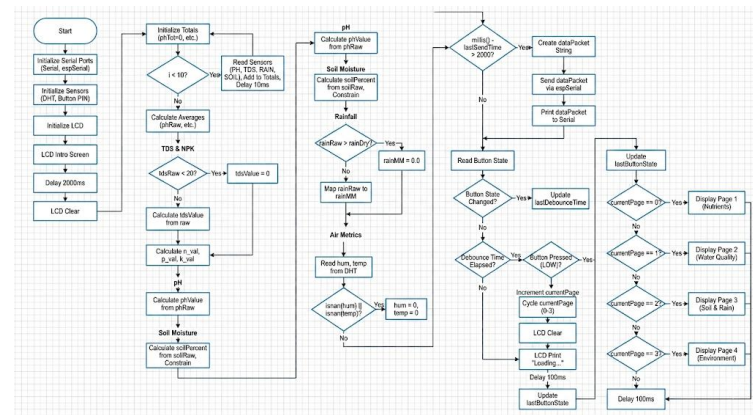


Fig. 21. Master Node (Arduino) Flowchart

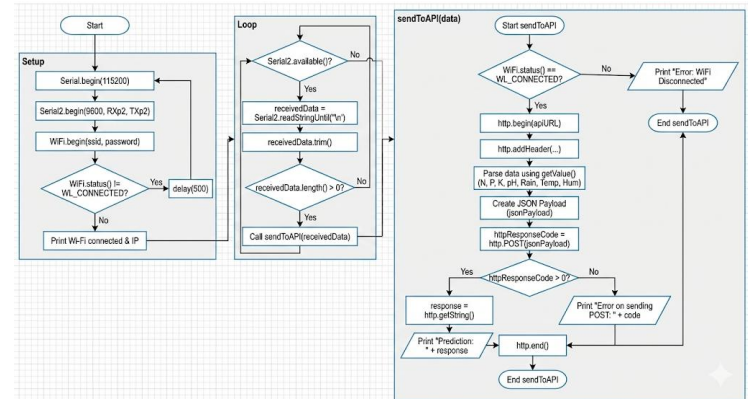


Fig. 22. Slave Node (ESP32) Flowchart

X. Machine Learning Model and Web Platform

A. Machine Learning Integration

A Random Forest Classifier comprising 200 decision trees is employed for crop prediction. The model is trained using the *BanglaVeg* dataset, which includes soil nutrients, environmental parameters, and crop suitability labels specific to Bangladesh.

Input Features:

- Nitrogen (N), Phosphorus (P), Potassium (K)
- Soil pH
- Temperature
- Humidity
- Rainfall

Output:

- Predicted optimal crop with confidence scores

The trained model is deployed via a cloud-based API and integrated with Firebase to provide real-time recommendations.

B. Web Dashboard

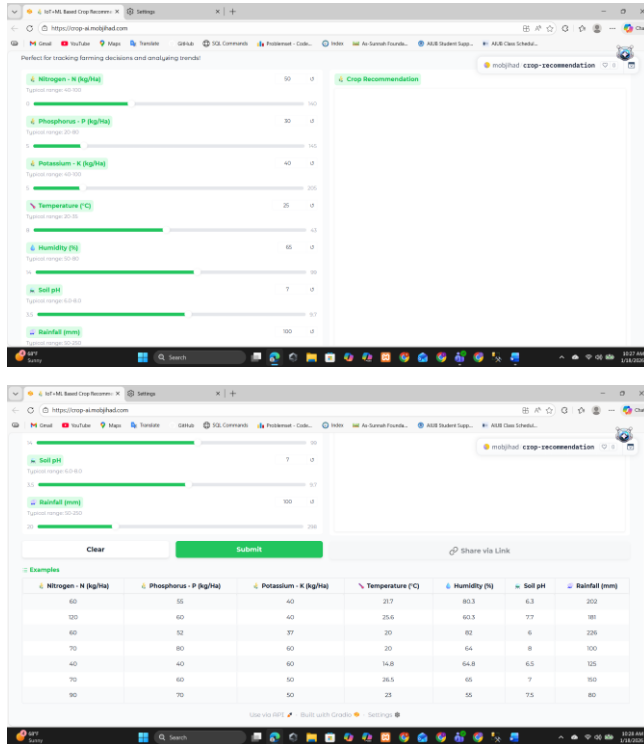


Fig. 23. System Web Interface

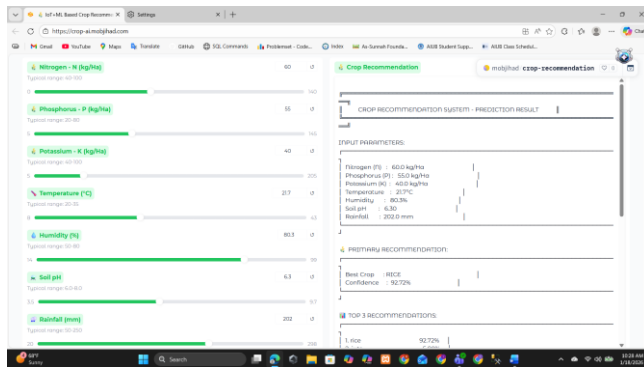


Fig. 24. System Recommending Crops

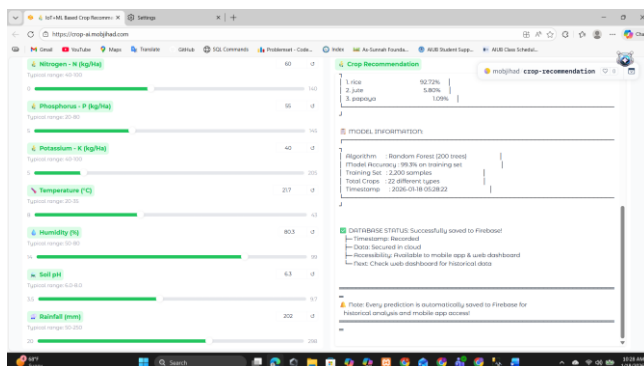


Fig. 25. System Model Information Panel

C. Mobile Application Interface

The mobile application allows farmers to submit personal information and land location data, which are associated with

sensor readings for personalized recommendations.

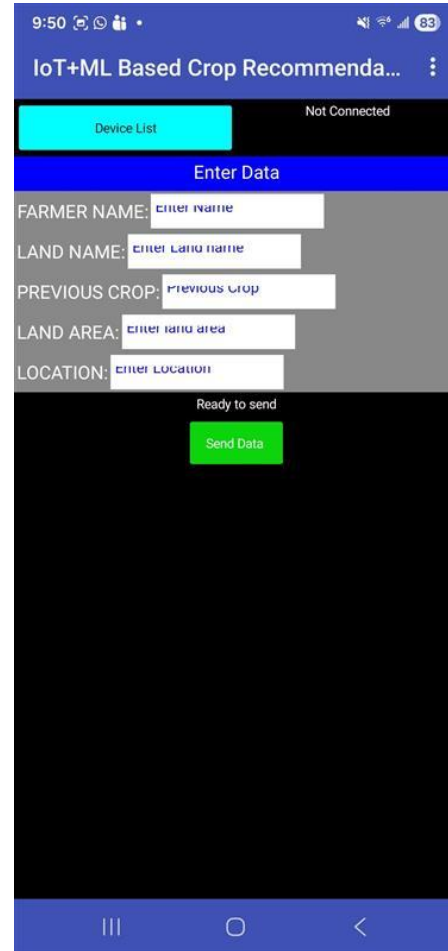


Fig. 26. Sending Personal Information and Location of Farmer to the Website

XI. Results and Discussions

This section presents the discussion and analysis of results obtained from simulation and experimental validation of the proposed system. Detailed hardware implementation and measured responses are presented in Section VII (Experimental Setup and Hardware Results), while detailed simulation results with Proteus circuit diagrams and LCD output images are provided in Section VIII (Simulation Setup – Proteus).

A. Simulation / Numerical Analysis

The proposed system was first validated through simulation using **Proteus Design Suite** to verify circuit behavior and firmware logic prior to hardware implementation. Virtual sensor models were used to emulate pH, TDS, soil moisture, temperature, humidity, and rainfall inputs.

The complete simulation setup, circuit diagrams, and LCD output results are presented in Section VIII (Simulation Setup – Proteus).

The simulation confirmed correct analog-to-digital conversion, serial communication between the Arduino and ESP32, and proper LCD data visualization. Numerical calculations for EC conversion, NPK estimation, and pH calibration produced stable and repeatable outputs under varying simulated input conditions. These results demonstrate the correctness of the implemented mathematical models and system logic before real-world deployment.

B. Measured Response / Experimental Results

The physical prototype was tested using real soil samples prepared with the **1:2 soil-to-distilled-water slurry method**. Sensor readings were successfully acquired and displayed on the LCD interface, including NPK values, pH, soil moisture, rainfall intensity, temperature, and humidity.

The complete experimental hardware setup, LCD display outputs, and system operation images are detailed in Section VII (Experimental Setup and Hardware Results).

The ESP32 reliably transmitted processed data to the Firebase cloud platform over Wi-Fi. Experimental results showed consistent sensor responses across repeated measurements, validating the effectiveness of the slurry-based sensing approach. The system operated stably in both online and offline monitoring modes.

C. Comparison between Numerical and Experimental Results

A comparison between simulated (numerical) results and experimental measurements indicates a close correlation in system behavior. Minor variations were observed due to environmental noise, sensor tolerance, and soil heterogeneity during real-world testing. However, the overall trends of pH, EC-derived NPK values, and moisture levels remained consistent across both **simulation results (Section VIII)** and **experimental results (Section VII)**.

This confirms that the simulation model accurately represents the real system and that the implemented algorithms are suitable for practical agricultural applications.

D. Cost Analysis

Cost efficiency was a primary design objective of the proposed system. By utilizing **low-cost water-type sensors** instead of expensive industrial soil sensors and adopting a slurry-based measurement approach, the overall system cost was significantly reduced. The use of open-source hardware platforms such as **Arduino Uno and ESP32** further minimized expenses.

Compared to conventional laboratory-based soil testing methods, the proposed system offers an affordable, reusable, and scalable solution suitable for smallholder farmers in Bangladesh.

E. Limitations of the Project

Despite its advantages, the proposed system has several limitations. The EC-based NPK estimation provides a **relative soil fertility index** rather than precise chemical concentrations. Sensor accuracy may be affected by temperature variations, soil composition differences, and long-term calibration drift. Additionally, the current system relies on Wi-Fi connectivity, which may be unreliable in remote rural areas. These limitations indicate potential areas for future improvement.

XII. Conclusion and Future Endeavors

This project presents a low-cost, IoT-enabled smart crop recommendation system tailored for the agricultural context of Bangladesh. By integrating slurry-based soil analysis, affordable sensors, cloud-based machine learning, and a Random Forest classifier trained on localized data, the system provides practical and data-driven farming recommendations. **Both simulation results (Section VIII) and experimental results (Section VII) confirm the reliability, affordability, and scalability of the proposed solution.**

Future endeavors include integrating long-range communication technologies such as **LoRaWAN** for remote area deployment, incorporating renewable energy sources for autonomous operation, enhancing NPK estimation accuracy using hybrid sensing techniques, and extending the machine learning model to include pest and disease prediction. These improvements will further strengthen the system's contribution to sustainable agriculture and food security.

References (IEEE Style)

- [1] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] M. Amin, S. Rahman, and T. Islam, "IoT-based smart agriculture system for crop recommendation," *IEEE Access*, vol. 13, pp. 11245–11256, 2025.
- [3] M. Rahman, A. Hossain, and K. Ahmed, "Multi-modal data fusion for precision agriculture," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3210–3221, Apr. 2023.
- [4] Eutech Instruments, "TDS and conductivity relationship," *Thermo Fisher Scientific*, 2023. [Online]. Available: <https://www.thermofisher.com/bd/en/home/industrial/water-analysis/water-analysis-learning-center/tds-conductivity.html>
- [5] S. R. Sarker *et al.*, "BanglaVeg: A localized dataset for crop recommendation in Bangladesh," in *Proc. Int. Conf. Smart Agric. Technol.*, Dhaka, Bangladesh, 2025, pp. 55–60.

Appendix

- **Appendix A: Master Node Firmware (Arduino Uno)**

This appendix presents the complete firmware used in the **Master Node (Arduino Uno)**. The code is responsible for sensor data acquisition, signal calibration, LCD display control, and serial communication with the ESP32 Slave node.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <SoftwareSerial.h>

// ----- Pin Definitions -----
#define PH_PIN A0
#define TDS_PIN A1
#define SOIL_PIN A2
#define RAIN_PIN A3
#define DHT_PIN 2
#define BUTTON_PIN 4

// ----- Object Initialization -----
SoftwareSerial espSerial(10, 11); // RX, TX
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHT_PIN, DHT22);

// ----- Global Variables -----
float tdsValue = 0;
float ecValue = 0;
float pHValue = 0;
float n_val = 0, p_val = 0, k_val = 0;
float temperature = 0, humidity = 0;
int soilMoisture = 0;
int rainValue = 0;
int page = 0;

// ----- Setup Function -----
void setup() {
  Serial.begin(9600);
  espSerial.begin(9600);

  dht.begin();

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  lcd.init();
  lcd.backlight();

  lcd.setCursor(0,0);
  lcd.print(" Smart Crop AI ");
  lcd.setCursor(0,1);
  lcd.print(" Initializing ");
  delay(2000);
  lcd.clear();
}

// ----- Main Loop -----
void loop() {

  // ----- Read Sensors -----
  int tdsRaw = analogRead(TDS_PIN);
  int pHRaw = analogRead(PH_PIN);
  int soilRaw = analogRead(SOIL_PIN);
  rainValue = analogRead(RAIN_PIN);

  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  // ----- TDS to EC Conversion -----
  tdsValue = (tdsRaw * 5.0 / 1023.0) * 1000;
  ecValue = tdsValue / 0.64;

  // ----- NPK Estimation -----
  n_val = 0.50 * ecValue;
  p_val = 0.20 * ecValue;
  k_val = 0.35 * ecValue;

  // ----- pH Calculation -----
  float pHVoltage = pHRaw * 5.0 / 1023.0;
  pHValue = 7.0 + ((2.5 - pHVoltage) / 0.18);
```

```

// ----- Soil Moisture -----
soilMoisture = map(soilRaw, 1023, 0, 0, 100);

// ----- LCD Display Control -----
if (digitalRead(BUTTON_PIN) == LOW) {
  page++;
  if (page > 3) page = 0;
  delay(300);
}

lcd.clear();
if (page == 0) {
  lcd.print("N:");
  lcd.print(n_val);
  lcd.print(" P:");
  lcd.print(p_val);
  lcd.setCursor(0,1);
  lcd.print("K:");
  lcd.print(k_val);
}
else if (page == 1) {
  lcd.print("pH:");
  lcd.print(phValue);
  lcd.setCursor(0,1);
  lcd.print("TDS:");
  lcd.print(tdsValue);
}
else if (page == 2) {
  lcd.print("Soil:");
  lcd.print(soilMoisture);
  lcd.print("%");
  lcd.setCursor(0,1);
  lcd.print("Rain:");
  lcd.print(rainValue);
}
else if (page == 3) {
  lcd.print("Temp:");

```

```

  lcd.print(temperature);
  lcd.setCursor(0,1);
  lcd.print("Hum:");
  lcd.print(humidity);
}

// ----- Send Data to ESP32 -----
String dataPacket = String(n_val) + "," + String(p_val) +
", " + String(k_val) + ", " +
String(phValue) + ", " + String(soilMoisture) +
", " +
String(temperature) + ", " + String(humidity);

espSerial.println(dataPacket);

delay(2000);
}

```

• Appendix B: Slave Node Firmware (ESP32)

This appendix presents the firmware used in the **Slave Node (ESP32)**. The ESP32 receives sensor data from the Arduino via UART communication and uploads the processed data to the **Firestore Realtime Database** over Wi-Fi.

```

#include <WiFi.h>
#include <FirebaseESP32.h>

// ----- WiFi Credentials -----
#define WIFI_SSID "Your_WiFi_Name"
#define WIFI_PASSWORD "Your_WiFi_Password"

// ----- Firebase Credentials -----
#define FIREBASE_HOST "your-project-
id.firebaseio.com"
#define FIREBASE_AUTH "your-database-secret"

// ----- UART Pins -----
#define RXD2 16
#define TXD2 17

```

```

FirebaseData firebaseData;

```

```

// ----- Setup Function -----
void setup() {
  Serial.begin(9600);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
  Firebase.reconnectWiFi(true);
}

// ----- Main Loop -----
void loop() {

  if (Serial2.available()) {
    String data = Serial2.readStringUntil('\n');

    // Split received data
    float N = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float P = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float K = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float pH = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float soil = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float temp = data.substring(0, data.indexOf(',')).toFloat();
    data = data.substring(data.indexOf(',') + 1);
    float hum = data.toFloat();

    // Upload to Firebase
    Firebase.setFloat(firebaseData, "/Sensors/Nitrogen", N);
    Firebase.setFloat(firebaseData, "/Sensors/Phosphorus",
P);
    Firebase.setFloat(firebaseData, "/Sensors/Potassium", K);
    Firebase.setFloat(firebaseData, "/Sensors/pH", pH);
    Firebase.setFloat(firebaseData, "/Sensors/SoilMoisture",
soil);
    Firebase.setFloat(firebaseData, "/Sensors/Temperature",
temp);
    Firebase.setFloat(firebaseData, "/Sensors/Humidity",
hum);
  }

  delay(1000);
}

```